

Assignment 3

MARS Fundamentals

Five assembly programs, verified output, and register/memory traces.

Review draft. The reflection is an AI-assisted draft to personalize. Check your syllabus collaboration policy before submitting. No submission to Canvas has been made.

Run the files on this Mac

1. Double-click **Launch MARS.command** in the parent outputs folder. Java 17 and MARS 4.5 are installed.
2. Choose **File > Open** and open one of the five .asm files in the assignment3 folder.
3. Under Settings, enable extended (pseudo) instructions and formats. Use the Default memory configuration and turn delayed branching off for these traces.
4. Choose **Run > Assemble**, then **Run > Go**. Read output in Run I/O. Run each program separately.
5. To trace: assemble again to reset, choose **Run > Step**, and watch the Registers and Data Segment panels. Turn off hexadecimal values to compare decimal register values below. Addresses below remain hexadecimal.

File	Expected output / result
01_hello_world.asm	Hello World!
02_print_1_to_100.asm	100 lines: integers 1 through 100
03_sum_even.asm	2550, also stored in sum
04_add_from_memory.asm	15; input words 8 and 7, result word 15
05_fizzbuzz.asm	FizzBuzz from 1 through 100 (range assumed from assignment context)

Instructions, pseudo-instructions, and directives

Basic instructions used: add, addiu, addu, slt, beq, bne, j, sll, lw, sw, div (two-register form), mfhi, syscall. The basic instruction **sll \$zero, \$zero, 0** does nothing and makes branch delay slots harmless if delayed branching is enabled.

Pseudo-instructions: li loads a constant, la loads an address, and move copies a register. The assembler translates them into basic instructions. For example, move \$a0, \$t0 becomes addu \$a0, \$zero, \$t0. In the Execute tab, compare Source with Basic; la may expand to multiple instructions. One Step executes one basic instruction, so source lines may repeat in a trace.

Assembler directives: .data, .text, .word, .ascii, and .globl guide assembly or allocate data; they are not CPU instructions. Syscalls use \$v0 for the service and \$a0 for the argument: 1 prints an integer, 4 prints a null-terminated string, and 10 exits.

1. Hello World

Actual MARS output: He l l o w o r l d ! followed by a newline. The .asciiz directive includes the terminating zero byte required by print-string syscall 4.

Step	PC before	Instruction	\$a0	\$v0
2	0x00400004		268500992	0
3	0x00400008	li \$v0, 4	268500992	4
4	0x0040000c	syscall	268500992	4
6	0x00400014	syscall	268500992	10

The string starts at 0x10010000 (268500992). \$a0 points to that string. Changing \$v0 to 10 selects exit; it does not change the message in memory.

2. Print integers 1 through 100

\$t0 holds the current integer, \$t1 holds the inclusive upper bound, and \$t2 is the comparison result. Each iteration prints \$t0 and a newline, then increments \$t0. The exit check prevents printing 101.

Step	PC before	Instruction	\$t0	\$t1	\$t2
3	0x00400008	slt \$t2, \$t1, \$t0	1	100	0
13	0x00400030	addiu \$t0, \$t0, 1	2	100	0
1191	0x00400008	slt \$t2, \$t1, \$t0	100	100	0
1203	0x00400008	slt \$t2, \$t1, \$t0	101	100	1

Complete output. Each cell is one successive output line; read down each column, then move right.

Lines 1-20	21-40	41-60	61-80	81-100
1	21	41	61	81
2	22	42	62	82
3	23	43	63	83
4	24	44	64	84
5	25	45	65	85
6	26	46	66	86
7	27	47	67	87
8	28	48	68	88
9	29	49	69	89
10	30	50	70	90
11	31	51	71	91
12	32	52	72	92
13	33	53	73	93
14	34	54	74	94
15	35	55	75	95
16	36	56	76	96
17	37	57	77	97
18	38	58	78	98
19	39	59	79	99
20	40	60	80	100

3. Sum of the even integers

Actual MARS output: 2550. The loop starts at 2 and increases by 2, so it adds $2 + 4 + \dots + 100$. There are 50 terms: $50 \times (2 + 100) / 2 = 2550$.

Step	PC before	Instruction	\$t0	\$t2	\$t3
7	0x00400018	add \$t2, \$t2, \$t0	2	2	0
13	0x00400018	add \$t2, \$t2, \$t0	4	6	0
301	0x00400018	add \$t2, \$t2, \$t0	100	2550	0
304	0x0040000c	slt \$t3, \$t1, \$t0	102	2550	1

After `sw $t2, 0($t4)`, step 308, the word at **0x10010000** changes from 0 to **2550**. \$t2 remains 2550. The final current integer is 102, which triggers exit.

4. Load, add, and store memory values

Actual MARS output: 15. Following the offset pattern in instructor example `mips5.asm`, \$t3 is the base address; `lw` uses offsets 0 and 4, and `sw` writes offset 8. All words are aligned on four-byte boundaries.

Step	PC before	Instruction	\$t0	\$t1	\$t2
3	0x00400008	lw \$t0, 0(\$t3)	8	0	0
4	0x0040000c	lw \$t1, 4(\$t3)	8	7	0
5	0x00400010	add \$t2, \$t0, \$t1	8	7	15
6	0x00400014	sw \$t2, 8(\$t3)	8	7	15

Memory label	Address	Before	After store
first	0x10010000	8	8
second	0x10010004	7	7
result	0x10010008	0	15

The `lw` instructions copy values from memory into registers; they do not erase the memory words. The `add` instruction changes \$t2, while `sw` changes only the result word in memory. Signed `add` raises an exception when the mathematical sum is outside the signed 32-bit range.

5. FizzBuzz

Range: 1 through 100. \$t0 is the current integer; \$t2 = 3 and \$t3 = 5. Two-register div writes its remainder to HI; mfhi copies each remainder to \$t4 or \$t5. A zero remainder means divisibility. Both zero prints FizzBuzz; only \$t4 zero prints Fizz; only \$t5 zero prints Buzz; otherwise print the integer.

Step	PC before	Instruction	\$t0	\$t4	\$t5
11	0x00400028	mfhi \$t5	1	1	1
51	0x00400028	mfhi \$t5	3	0	3
92	0x00400028	mfhi \$t5	5	2	0
293	0x00400028	mfhi \$t5	15	0	0
1996	0x00400028	mfhi \$t5	99	0	4
2017	0x00400028	mfhi \$t5	100	1	0

Complete output. Each pair gives the input integer and its actual output. Read down each pair, then move right.

n	Output	n	Output	n	Output	n	Output
1	1	26	26	51	Fizz	76	76
2	2	27	Fizz	52	52	77	77
3	Fizz	28	28	53	53	78	Fizz
4	4	29	29	54	Fizz	79	79
5	Buzz	30	FizzBuzz	55	Buzz	80	Buzz
6	Fizz	31	31	56	56	81	Fizz
7	7	32	32	57	Fizz	82	82
8	8	33	Fizz	58	58	83	83
9	Fizz	34	34	59	59	84	Fizz
10	Buzz	35	Buzz	60	FizzBuzz	85	Buzz
11	11	36	Fizz	61	61	86	86
12	Fizz	37	37	62	62	87	Fizz
13	13	38	38	63	Fizz	88	88
14	14	39	Fizz	64	64	89	89
15	FizzBuzz	40	Buzz	65	Buzz	90	FizzBuzz
16	16	41	41	66	Fizz	91	91
17	17	42	Fizz	67	67	92	92
18	Fizz	43	43	68	68	93	Fizz
19	19	44	44	69	Fizz	94	94
20	Buzz	45	FizzBuzz	70	Buzz	95	Buzz
21	Fizz	46	46	71	71	96	Fizz
22	22	47	47	72	Fizz	97	97
23	23	48	Fizz	73	73	98	98
24	Fizz	49	49	74	74	99	Fizz
25	Buzz	50	Buzz	75	FizzBuzz	100	Buzz

At completion \$t0 = 101, \$t6 = 1, and no line is printed for 101. For 100, remainders are 1 and 0, so the final output line is Buzz.

Verification and edge cases

Tests executed with the installed official MARS 4.5 JAR. Each complete output was compared with an independent expected result. Temporary edge-case copies change only the upper-bound constant or input .word values; the five delivered files retain the assignment values.

Program / case	Input / setting	Observed result	Status
All programs	Default and delayed branching	Identical correct output	PASS
Print loop	Upper bounds 0, 1, 2, 99, 100	All output matches independent calculation	PASS
Even sum	Upper bounds 0, 1, 2, 99, 100	All output matches independent calculation	PASS
FizzBuzz	Upper bounds 0, 1, 2, 99, 100	All output matches independent calculation	PASS
Memory add	8 + 7	15	PASS
Memory add	0 + 0	0	PASS
Memory add	-8 + 7	-1	PASS
Memory add	-8 + -7	-15	PASS
Memory add	2147483647 + 0	2147483647	PASS
Memory add	-2147483648 + 0	-2147483648	PASS
Memory add	2147483647 + 1	MARS arithmetic overflow exception (expected)	PASS
FizzBuzz	1, 3, 5, 15, 99, 100	1, Fizz, Buzz, FizzBuzz, Fizz, Buzz	PASS

Trace method. Tables show actual state captured after executing the listed basic instruction through MARS simulateStepAtPC. PC before identifies that instruction. The trace runner uses the same MARS assembler, simulator, registers, and memory as the GUI, with Default memory layout and delayed branching disabled. These are recorded MARS state tables, not GUI screenshots. GUI steps on page 1 reproduce the same states.

One-paragraph reflection draft - personalize before submission

Working through these programs shows how registers, memory, and control flow interact in MIPS. A register holds a current value such as a loop counter or running total, while `lw` and `sw` explicitly transfer word values between memory and registers. The print loop stops when its counter exceeds the upper bound, and the even-number loop changes the counter by two to include only even values. FizzBuzz uses the remainder in `HI` to choose the appropriate branch. Comparing source lines with assembled basic instructions also shows that `li`, `la`, and `move` are assembler conveniences rather than individual MIPS instructions, while directives such as `.word` define data. Stepping through the recorded states makes it easier to see exactly when a register changes and when a value is stored in memory.

Sources and collaboration acknowledgement

Instructor: Dominic Dabish, CS240-01 Fall 2026, Canvas Code folder, Thursday examples `mips1.asm` through `mips15.asm`. Memory-offset design follows `mips5.asm`; printing and looping concepts follow `mips7-mips13.asm`. The delivered files are separate commented implementations with bounds checks and explicit exit services.
<https://sdsu.instructure.com/courses/212306/files/folder/Code>

Software and reference: Pete Sanderson and Kenneth Vollmar, MARS 4.5 official distribution, bundled source, command-line help, and `syscall` definitions.
<https://github.com/dpetersanderson/MARS>

OpenAI Codex assisted with setup, reading the instructor examples, creating the programs, automated MARS testing and trace capture, and drafting this report and reflection. The student should review and understand the work, personalize the reflection, and follow the course collaboration policy. The syllabus policy was not verified during this preparation.